

SkillROOT Mobile

This project incorporates AI agent-driven workflows to manage complex infrastructure transitions. Specifically, advanced agent tools were utilized to execute the port-over of the Identity Provider (IDP) authentication system, ensuring a seamless and secure integration between Firebase Auth and the SkillROOT API Portal.

Firebase Configuration

To set up or update the Firebase configuration for this project, you need to use the FlutterFire CLI.

1. Prerequisites

Before configuring FlutterFire, ensure you have the Firebase CLI installed and are logged in:

- **Install Firebase CLI:** `npm install -g firebase-tools`
- **Login:** `firebase login`

2. Install FlutterFire CLI

Run the following command to install the FlutterFire CLI globally:

```
dart pub global activate flutterfire_cli
```

3. Run Configuration

Execute the configuration command from the root of the project:

```
flutterfire configure
```

- When prompted, select the Firebase project: **humequity-idp (Humequity-IDP)**.
- This will automatically generate `lib/firebase_options.dart` and update your native Android/iOS configuration files.

[!IMPORTANT]

Authentication & Access: Please ensure you are using your **npowell@technicate.com** for the `firebase login` step. If you find that you do not have access to the project or lack the necessary permissions to run the configuration, reach out to **elopez@humequity.com** for assistance.

Firebase & Authentication Implementation

This implementation was facilitated by AI agent-driven workflows to execute the port-over of the Identity Provider (IDP) authentication system. This ensures a robust authentication lifecycle that integrates Firebase Auth with the SkillROOT API Portal. Below is a summary of the key changes made to ensure a secure and resilient user experience:

1. Hybrid Authentication Flow

- **Client-Side:** Uses `firebase_auth` for initial user credential verification.
- **Server-Side Verification:** Once Firebase authenticates the user, the app retrieves a fresh **ID Token** and sends a **POST** request to the backend portal at the `/v1/validate-token` endpoint.
- **Bearer Authentication:** All subsequent API calls automatically include the Firebase ID Token in the `Authorization: Bearer <token>` header (replacing previous body-parameter based authentication).
- **Asset Attribution:** *By using this approach, we will always know what credentials are using what assets.*

2. User Registration Workflow

- **Firebase-First Creation:** The registration flow now uses `FirebaseAuth.instance.createUserWithEmailAndPassword` directly in the client. This ensures that users are immediately provisioned in the Identity Provider.
- **Client-Side Validation:** Leveraging Firebase's built-in error handling for weak passwords, malformed emails, and duplicate accounts.
- **Portal Sync Active:** The app now successfully notifies the SkillROOT API Portal (`/v1/register/user`) with the user's profile details (Names, Phone) immediately after Firebase creation, ensuring the SkillROOT database is kept in sync.
- **ID Token Verification:** The backend verifies the Firebase ID Token provided in the `Authorization` header to secure the registration endpoint.

3. API Service Enhancements (`lib/services/api.dart`)

- **Resilience:** Added a 10-second timeout to all network requests to prevent UI hangs.
- **Error Handling:** Implemented comprehensive try-catch blocks to handle `SocketException`, `TimeoutException`, and invalid JSON formats.
- **Flexible Success Logic:** The API client now recognizes both standard `{"success": true}` and HTTP-style `{"status": 200}` response formats from the portal.
- **Diagnostic Logging:** Added verbose debug prints (prefixed with `DEBUG:`) to the console to track Firebase status, network status, and raw server responses.

4. UI Resilience & Hang Prevention

- **Resilient Loaders:** Refactored `LoginScreen`, `RegisterScreen`, and `ResetScreen` using try-catch-finally blocks. This ensures that the loading spinner is **always** dismissed, even if a network or authentication error occurs.
- **Variable Scoping:** Fixed issues with undeclared result variables to ensure smooth state transitions.

4. Complete Logout & State Clearing

- **Global Reset:** Implemented a `Provider.reset()` method that wipes all cached in-memory data (User info, Jobs, Training lists) to prevent data leakage between sessions.
- **Session Clearance:** The logout flow now sequentially signs out of Firebase, clears the local `Api` session cookie, resets the `Provider` state, and redirects the user to the login screen.
- **Unified Entry Points:** Integrated the full logout sequence into both the **Profile Screen** and the **Sidebar Drawer Menu**.

5. Emulator & Dev Environment

- **Automatic Host Detection:** The `lib/config.dart` now uses a helper to automatically route requests to `10.0.2.2` for Android emulators and `localhost` for Web/iOS, simplifying cross-platform development.